

Aligned Standards:

- **CCSS.ELA-Literacy.W.11-12.6:** Use technology, including the Internet, to produce and publish writing and to interact and collaborate with others.
- **ISTE Standards for Students 5: Computational Thinkers** - Students develop and employ strategies for understanding and solving problems that leverage technology's power.

Objective: Create an application that allows users to view the 5-day weather forecast. The app's UI should enable the user to enter the ZIP code and click a button to display the weather. The editText and "run" button will be the only two required input widgets. The weather app should be based on a chosen theme. This means that your app should display the weather and revolve around the selected theme. Put thought into the color scheme and aesthetic, as most downloaded apps do not simply feature black text on a white background.

The app should include a feature that allows users to input a ZIP code. When the user inputs a location, the app will display the current weather conditions and a relevant quote based on the theme. For example, "*Phyllis: It's raining cats & dogs out there*" could be displayed for a rainy day, while "*Dwight: The greatest snowball isn't a snowball at all. It's fear*" could be displayed for a snowy day.

At the bottom of the application, display an image, temperature, a short description, and the time for each weather report for the next five days. OpenWeather makes it easy to obtain weather data for five days. The first panel in the 5-day forecast should represent the current day or time. The exact layout for this information is up to you, but you need to display the requested information.

Note: Part of this project's criteria, and projects in the future, will be your ability to think critically and work independently. Read the directions carefully. Major teacher assistance will result in a project grade deduction. If you are unsure of an error, Google it! Since things are always changing in technology, your ability to read up on something, follow a tutorial, and/or troubleshoot independently are imperative skills to be successful in the field. Challenge yourself! Maybe try talking it through with your programming duck. 🦆

PART 1 - Getting Started: Go to [OpenWeatherMap](#) and create an account before you do anything. You should then go to [How to Start with OpenWeather API](#) and read about how to make a request for the weather.

OpenWeatherMap Application Guide

CS Topics: Mobile Application Development

The OpenWeather website provides several examples of weather information based on ZIP codes. **In this app, you must use the [GeoCoding API](#) call to give the latitude and longitude based on a given ZIP code. The latitude and longitude will then retrieve the weather data using the [5-Day / 3-Hour Forecast API](#).** You will display the latitude, longitude, and city/town when the user requests the weather via the zip code. **You may not use any other service to provide this information.**

Before Continuing: Once you are registered, you should be able to get a response from your API request in JSON format. **Do not go on until you do.** You can test your key and that you receive a request using one of the API calls from the site. ***Test the link to your API key in a web browser before you try it in your application to ensure it works!*** If you are not getting a formatted JSON response with the complete data points, something went wrong, and you need to figure out what it is.

Test Link: https://api.openweathermap.org/geo/1.0/zip?zip=08852.us&appid={API_KEY}

(replace {API_KEY} with your OpenWeatherKey for testing, brackets not included in actual URL.)

AsyncTask Starter Code:

Java

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        AsyncThread task = new AsyncThread();
        task.execute();
    }
    public class AsyncThread extends AsyncTask<Void, Void, Void>{
        @Override
        protected Void doInBackground(Void... voids) {
            // This is where you will download your data.
            // You will need to override another method to update the
            UI
            Log.d("TAG", "Thread ");
            return null;
        }
    }
}
```

PART 2 - Retrieving the Data: Go into the **AndroidManifest.xml** and add the user permission to access the internet. You will have to do a Google search to find out how to add this permission. After, use the AsyncTask Class (starter code given above) to create and run a background thread to download the current weather information to your application. When your task runs, you must do a few things to download the information.

Here are the steps you need to follow:

1. Create an instance of the **URL** class and call the constructor. The constructor takes a **String**, so you should send it an API call you create using an example from the [GeoCoding API](#) for ZIP codes (other options are available, but you must use the call with ZIP codes). Feel free to place a known ZIP code in this call, such as 08852, and set the country code to US. You will eventually update this URL in Part 3 so that it can use user input to retrieve the zip code.
 - **NOTE:** As you progress, you will likely encounter permission problems. Your app will need permission to use the internet, so you must Google how to add this to your app.
2. Declare an instance of the **URLConnection** class and assign it to the value you get from calling the **openConnection** method on the **URL** instance from Step 1.
3. Declare an instance of the **InputStream** class and assign it the value you get from calling the **getInputStream** method from the **URLConnection** instance from Step 2.
4. Create a **BufferedReader**, and when you initialize its constructor, send it a new **InputStreamReader**. The constructor for the **InputStreamReader** should be sent the **InputStream** you created in Step 3.
5. Read the information using the **BufferedReader** and save it into a single **String**. That String will contain all of the weather data. You can use this **String** to construct a JSON data type that will allow you to access the necessary data points for this project as you did in the previous JSON practice. (You may first want to test the String in a Log statement before setting up the JSON

OpenWeatherMap Application Guide

CS Topics: Mobile Application Development

structure.) Select the correct data type for the JSON structure, or it will result in an error.

BEFORE CONTINUING: Ensure that your code is working correctly at this point. At the minimum, you should be able to create a Log statement that displays some of the retrieved JSON data points. You should test a ZIP code by hardcoding it into the API call along with your country code (US) and API key. Remember that you will make two API calls, first to the Geocoding API and then to the 5-Day/3-Hour Forecast API, so you may have to repeat portions of this process to get both components working together. Start and test the Geocoding API before coding the second API call for the weather data. A sample output for the GeocodingAPI is given below for your testing purposes.

Sample Output for Geocoding Log with ZIP code 08852:

```
{"zip": "08852", "name": "Middlesex County", "lat": 40.3869, "lon": -74.5558, "country": "US"}
```

PART 3 - Cleaning up AsyncTask: The AsyncTask should be sent a reference to the String that contains the ZIP or city/state information (from the EditText). In the starter code, the AsyncTask isn't sent anything. Instead of <Void, Void, Void>, one of the parameters will be a String. You will need to do a little research to find out which one. The app will work fine without sending it, but if the AsyncTask was not an inner class, it wouldn't have access to the instance variables from the parent class and would not work correctly. However, as a requirement for this project, you must write your code so it is explicitly sent to the given String. You should comment next to the String used in the AsyncTask, so it is easier to grade.

Note: You must get the latitude and longitude with the GeoCoding API call, which can be executed via a button click. You can create a second button to make a second API call to get the weather with the given latitude/longitude retrieved from the Geocoding API. You will also have the option to display all the information at once with only a single button click. Since it is

08852 SEARCH

Phyllis: The plants are gonna love this.

Latitude: 40.3869°, Longitude: 40.3869°
Location: Middlesex County, US



Current Temperature: 32.31°

02-13	 clear sky	Min: 27.52° Max: 32.31°
02-14	 clear sky	Min: 20.08° Max: 20.08°
02-15	 overcast clouds	Min: 36.66° Max: 36.66°
02-16	 overcast clouds	Min: 31.33° Max: 31.33°
02-17	 overcast clouds	Min: 27.95° Max: 27.95°

11787 SEARCH

*Michael: Clouds... means it's gonna rain.
Dwight: If they were altocumulus.
Not cirrostratus...
Michael, get your clouds right!*

Latitude: 40.8542°, Longitude: 40.8542°
Location: Smithtown, US



Current Temperature: 33.75°

02-13	 few clouds	Min: 33.19° Max: 33.75°
02-14	 clear sky	Min: 27.37° Max: 27.37°
02-15	 light rain	Min: 39.31° Max: 39.31°
02-16	 overcast clouds	Min: 34.27° Max: 34.27°
02-17	 overcast clouds	Min: 32.47° Max: 32.47°

more difficult to use a single button to execute two different API calls in sequence, there will be 5 points on the rubric if you use only one button click to get the coordinates and weather. You will still be able to receive most of the points (95%) if you use two different button clicks.

FAQ:

- **How should I display the temperature?**
 - It must be Fahrenheit.
- **Will I lose points if I do not send the AsyncTask a String?**
 - Yes
- **I noticed an odd part of the code with "...". What is it?**
 - It is varargs. You will need to Google it! The idea is that you can research what you don't understand at this point. It is a really important skill in college and in computer science. If you have a question, I will point you in the right direction, but you should not expect me to answer it completely.

Starting Screen:



Invalid Zip Code

